

GA – Practica en clase | Formativa | Individual | Presencial | Unidad IV

TRAZADO DEL FLUJO COMPLETO

de una Peticion HTTP en Spring Boot 3 con IntelliJ IDEA

solicitud HTTP → DispatcherServlet → Security → Controller → Service → Repository →
PostgreSQL → JSON

Aplicaciones Web – Ingenieria de Software – UTEQ 2025-2026

Tipo	GA – Practica en clase
Modalidad	Individual – presencial en laboratorio
Duracion	90 minutos
Entregable	Ninguno – documentar en cuaderno o editor de texto durante la clase
Herramientas	IntelliJ IDEA (CE o Ultimate), proyecto Spring Boot 3 del PFC, Postman
Objetivo	Identificar las clases y metodos Java exactos donde ocurre cada paso del flujo MVC en el propio PFC, usando el depurador de IntelliJ IDEA

Que vamos a aprender

Cuando el cliente Angular hace `GET /api/v1/productos`, dentro de Spring Boot ocurren **9 pasos** en menos de 100 ms. La mayoría de los desarrolladores junior no los conocen porque el framework los ejecuta automaticamente. Esta practica los hace visibles uno a uno usando el depurador.

Al terminar la clase, cada estudiante tendra documentado el nombre exacto de la clase Java y el metodo donde ocurre cada paso en **su propio PFC**.

Fundamento teorico: el patron MVC en Spring Boot

Spring Boot no implementa MVC de la forma tradicional (Modelo-Vista-Controlador con vistas HTML). En el contexto de una API REST consumida por Angular, el patron se transforma:

Capa MVC	En Spring Boot REST	Que hace
M (Modelo)	Entidad JPA + DTO	Representa los datos del dominio; la entidad mapea a la tabla de PostgreSQL; el DTO es lo que viaja en el JSON
V (Vista)	Serializacion JSON	Jackson convierte el objeto Java a texto JSON automaticamente; no hay HTML
C (Controlador)	@RestController	Recibe la solicitud HTTP, valida parametros, delega al servicio y devuelve ResponseEntity
+ Capa de servicio	@Service	Logica de negocio; transacciones; no existe en el MVC clasico pero es esencial en Spring
+ Repositorio	JpaRepository	Acceso a datos; abstrae el SQL; habla directamente con PostgreSQL via Hibernate
+ Seguridad	JwtAuthFilter	Intercepta TODA solicitud antes de que llegue al controlador; valida el token JWT

El flujo completo: 9 pasos

El siguiente diagrama muestra la secuencia completa desde que el cliente Angular envia la solicitud hasta que recibe el JSON de respuesta. Cada paso corresponde a una clase Java real en su proyecto.

#	Componente	Clase / Interfaz Spring	Que ocurre
1	Cliente Angular	HttpClient + interceptor	Envia GET /api/v1/productos con Authorization: Bearer [token]
2	Tomcat embebido	TomcatEmbeddedWebappClassLoader	Recibe la conexión TCP; parsea el HTTP; crea HttpServletRequest
3	DispatcherServlet	DispatcherServlet.doDispatch()	Punto de entrada de Spring MVC; envia la solicitud al HandlerMapping
4	Cadena de filtros	JwtAuthFilter.doFilter()	Extrae el token JWT del encabezado; valida firma y expiración; consulta Redis (blacklist); establece el SecurityContext
5	HandlerMapping	RequestMappingHandlerMapping	Encuentra el metodo del controlador que coincide con la URL y el verbo HTTP
6	Controlador	@RestController.metodo()	Recibe los parametros ya mapeados; valida con @Valid; llama al servicio
7	Servicio	@Service.metodo()	Logica de negocio; abre la transacción (@Transactional); llama al repositorio
8	Repositorio + Hibernate	JpaRepository.findAll()	Genera el SQL; lo envia a PostgreSQL; mapea el ResultSet a entidades Java
9	Jackson + Tomcat	MappingJackson2HttpMessageConverter	Serializa la respuesta de DTOs a JSON; escribe la respuesta HTTP con código 200

Parte 1: Observar el flujo con el depurador de IntelliJ IDEA

El docente tiene el proyecto de demostración en el proyector. Sigán los pasos en sus propios proyectos del PFC en paralelo.

3.1 Configurar y arrancar en modo depuración

Abrir el proyecto en IntelliJ IDEA

2 minutos

1. Abrir IntelliJ IDEA. Ir a **File > Open** y seleccionar la carpeta **backend/** del repositorio del PFC.
2. Esperar a que Maven descargue las dependencias (barra de progreso en la parte inferior).
3. Verificar que el proyecto compila: ir al terminal integrado y ejecutar:

```
./mvnw compile      # Linux / Mac
mvnw.cmd compile    # Windows
```

El resultado esperado es **BUILD SUCCESS**. Si hay errores, avisarle al docente antes de continuar.

Colocar el primer breakpoint en el controlador

3 minutos

1. En el panel **Project** (izquierda), navegar hasta la clase del controlador principal del PFC. Por ejemplo: `src/main/java/ec/edu/uteq/[proyecto]/controller/ProductoController.java` (o el nombre real de su controlador).
2. Encontrar el método del endpoint GET que lista los recursos (p. ej. `listar()`, `getAll()`, `index()`).
3. Hacer clic en el **numero de línea** de la primera instrucción del método (no en la anotación `@GetMapping`). Debe aparecer un círculo rojo • indicando el breakpoint.

Atajo de teclado

En lugar de hacer clic en el número de línea, puede presionar **Ctrl+F8** (Windows/Linux) o **Cmd+F8** (Mac) con el cursor en la línea deseada.

Colocar el segundo breakpoint en el filtro JWT

2 minutos

1. Navegar hasta la clase `JwtAuthFilter.java` (generalmente en el paquete `security/`).
2. Buscar el método `doFilterInternal()` o equivalente.
3. Colocar un breakpoint en la primera línea donde se extrae el token del encabezado `Authorization`.

Colocar el tercer breakpoint en el servicio**1 minuto**

1. Navegar hasta la clase del servicio (p. ej. `ProductoService.java` o `ProductoServiceImpl.java`).
2. Colocar un breakpoint en la primera linea del metodo que sera llamado desde el controlador (p. ej. `listarTodos()`).

Arrancar el proyecto en modo Debug**2 minutos**

1. En la esquina superior derecha de IntelliJ IDEA, hacer clic en el icono del **insecto** (*Debug*) junto al boton de Play. O presionar **Shift+F9**.
2. Esperar a que aparezca en la consola: **Started [NombreApp] in X.XXX seconds**.
3. En la barra inferior, verificar que la pestana **Debug** esta activa.

Verificar que el servidor esta listo

Abrir el navegador y acceder a `http://localhost:8080/actuator/health`. Debe devolver `{"status":"UP"}`. Si devuelve error de conexion, el servidor aun no ha arrancado completamente.

3.2 Disparar la solicitud y trazar el flujo**Enviar la solicitud desde Postman****3 minutos**

1. Abrir Postman. Crear una nueva solicitud **GET** a la URL del endpoint de listado del PFC. Por ejemplo: `http://localhost:8080/api/v1/productos`
2. Agregar el encabezado de autenticacion: **Authorization: Bearer [token_valido]**. Para obtener un token valido, primero hacer **POST /api/v1/auth/login** con credenciales de prueba y copiar el **accessToken** de la respuesta.
3. Hacer clic en **Send**.
4. Postman quedara *esperando* la respuesta porque IntelliJ IDEA detendra la ejecucion en el primer breakpoint.

Observar el Paso 4: JwtAuthFilter**5 minutos**

IntelliJ IDEA se detiene en `JwtAuthFilter.doFilterInternal()`. En la pestana **Debug** de la parte inferior pueden ver:

- **Frames** (izquierda): la pila de llamadas completa. Observen las clases de Spring encima de su `JwtAuthFilter`: `ApplicationFilterChain`, `StandardWrapperValve`, `TomcatEmbedded...`. Eso es la **cadena de filtros de Tomcat y Spring** ejecutandose automaticamente.
- **Variables** (derecha): el objeto `request` con todos los detalles de la solicitud HTTP: URL, metodo, encabezados. Expandir `request` y buscar el encabezado `Authorization` para confirmar que el token llego.

Acciones en IntelliJ IDEA:

- **F8** (Step Over): ejecuta la linea actual y pasa a la siguiente sin entrar en metodos internos.
- **F7** (Step Into): entra dentro del metodo que se esta llamando.
- **F9** (Resume): continua la ejecucion hasta el proximo breakpoint.

Presionar **F8** varias veces para ver como se extrae el token, se valida con `JwtService.validateToken()`, y se establece el `SecurityContextHolder`. Observar el valor de la variable `username` cuando se extrae del token.

Presionar **F9** para continuar hasta el siguiente breakpoint (el controlador).

Observar el Paso 6: el Controlador

5 minutos

IntelliJ IDEA se detiene en el metodo del `@RestController`. Observar en el panel de Variables:

- Los parametros del metodo ya mapeados: si el endpoint tiene `@RequestParam`, `@PathVariable` o `Pageable`, ya estan disponibles como variables Java con sus valores.
- El objeto `this`: la instancia del controlador con sus dependencias inyectadas (el servicio aparece como campo).

Revisar el panel **Frames**: encima del controlador debe aparecer `RequestMappingHandlerAdapter.invokeHandlerMethod()` de Spring MVC. Ese es el **HandlerMapping** ejecutando el metodo del controlador.

Presionar **F7** para entrar en el metodo del servicio que llama el controlador.

Observar el Paso 7: el Servicio

5 minutos

IntelliJ IDEA esta ahora dentro del `@Service`. En el panel Frames pueden ver:

- **Proxies de Spring AOP**: si el servicio tiene `@Transactional`, encima del metodo del servicio apareceran clases como `TransactionInterceptor.invoke()` y `CglibAopProxy.intercept()`. Eso es Spring abriendo la transaccion de base de datos automaticamente antes de que el metodo se ejecute.
- El campo del repositorio inyectado: expandirlo para ver la implementacion de Hibernate encapsulada.

Presionar **F7** para entrar en el metodo del repositorio.

Observar el Paso 8: JpaRepository y Hibernate

5 minutos

Al entrar en el repositorio, entraran en codigo generado automaticamente por Spring Data JPA. Pueden ver:

- La clase `SimpleJpaRepository`: es la implementacion por defecto que Spring genera para todos los `JpaRepository`. Contiene el metodo `findAll()` que ustedes nunca escribieron pero que existe en tiempo de ejecucion.
- Si el IDE tiene configurado el log de SQL, en la consola de Spring Boot (pestanas inferiores) apareceran las sentencias SQL que Hibernate envia a PostgreSQL: `Hibernate: select p1_0.id, p1_0.nombre...`

Activar el log de SQL en el proyecto

En `application.yml` agregar:

```
spring.jpa.show-sql: true
spring.jpa.properties.hibernate.format_sql: true
logging.level.org.hibernate.SQL: DEBUG
```

Reiniciar el servidor en modo Debug para que el SQL aparezca en la consola.

Presionar **F9** para continuar hasta que Postman reciba la respuesta JSON.

Observar el Paso 9: la respuesta JSON*3 minutos*

1. Postman recibe la respuesta. Observar el JSON: es la lista de entidades Java serializada automaticamente por **Jackson**.
2. En IntelliJ IDEA, en la pestana **Console**, observar el log de la solicitud: tiempo total de procesamiento, SQL ejecutado y respuesta enviada.
3. Ir a **Run > View Breakpoints (Ctrl+Shift+F8)** para ver todos los breakpoints activos y desactivarlos o eliminarlos.

Parte 2: Documentar el flujo del propio PFC**Instruccion principal de la clase**

Completar la siguiente tabla con la informacion **real** del proyecto del PFC. No copiar nombres genericos: escribir el nombre exacto del archivo Java y el metodo donde ocurre cada paso en su propio proyecto. Esta documentacion es la evidencia de la practica.

Tabla de trazado de flujo — PFC: [Nombre del sistema]

#	Componente	Clase Java (nombre exacto)	Metodo (nombre exacto)	Paquete
1	Cliente Angular	<i>(No es Java; indicar el servicio Angular)</i>	<i>[NombreService].method()</i>	frontend/src
2	Tomcat embebido	<i>(Automatico, no editable)</i>	<i>Gestionado por Spring Boot</i>	<i>Spring interno</i>
3	DispatcherServlet	DispatcherServlet	doDispatch()	<i>Spring interno</i>
4	Filtro JWT	_____	_____	_____
5	HandlerMapping	RequestMappingHandlerMapping	getHandler()	<i>Spring interno</i>
6	Controlador	_____	_____	_____
7	Servicio	_____	_____	_____
8	Repositorio	_____	_____	_____

#	Componente	Clase Java	Metodo	Paquete
9	Serializacion JSON	MappingJackson2 HttpMessageConverter	write()	<i>Spring interno</i>

Como completar la tabla

Los pasos 1, 2, 3, 5 y 9 son iguales para todos porque son componentes internos de Spring Boot que no se modifican. Los pasos **4, 6, 7 y 8 son los que usted debe completar** con los nombres reales de sus clases. Para encontrar los nombres:

- **Paso 4:** buscar en el paquete `security/` la clase que extiende `OncePerRequestFilter`.
- **Paso 6:** buscar la clase anotada con `@RestController` que maneja el recurso principal del PFC.
- **Paso 7:** buscar la interfaz o clase anotada con `@Service` que usa el controlador del paso 6.
- **Paso 8:** buscar la interfaz que extiende `JpaRepository` y que usa el servicio del paso 7.

Parte 3: Preguntas de analisis

Responder en el cuaderno o editor de texto. El docente puede pedir compartir las respuestas al final de la clase.

1. En el panel Frames del depurador, cuando la ejecucion esta dentro del `JwtAuthFilter`, aparecen clases de Spring y Tomcat encima de la suya. ¿Que representa esa pila de llamadas? ¿En que orden se ejecutaron esas clases?

2. Cuando el depurador entra en el servicio, encima del metodo del servicio aparecen clases como `TransactionInterceptor` y `CglibAopProxy`. ¿Que esta haciendo Spring en ese momento? ¿Por que es importante que esas clases se ejecuten antes del metodo del servicio?

3. Al entrar en el repositorio, el depurador muestra la clase `SimpleJpaRepository` que nunca fue escrita por el equipo. ¿Como existe esa clase en tiempo de ejecucion si no esta en el codigo fuente del PFC? ¿Que mecanismo de Spring la genera?

4. Si se envia la solicitud GET **sin** el encabezado `Authorization: Bearer [token]`, ¿que paso del flujo detiene la solicitud y que codigo HTTP devuelve? ¿En que clase Java del proyecto ocurre esa decision?

5. Observando el SQL que Hibernate genera en la consola para el endpoint GET de listado: ¿Hibernate genera un SELECT con todos los campos de la tabla o solo los que el DTO necesita? ¿Que implicaciones de rendimiento tiene esto cuando la tabla tiene 50 columnas?

Errores frecuentes durante la practica

El breakpoint no se activa (icono rojo transparente o con cruz). Esto ocurre cuando el proyecto no ha compilado o cuando el archivo tiene errores de sintaxis. Ejecutar `./mvnw compile` y resolver los errores antes de arrancar en modo Debug.

Postman no recibe respuesta y el depurador no se detiene. Verificar que el servidor arranque en modo Debug (icono insecto, no triangulo Play) y que la URL en Postman coincide exactamente con la URL del endpoint del PFC incluyendo `/api/v1/`.

El depurador se detiene pero no se ve la variable esperada en el panel Variables. Algunas variables estan en el scope del metodo anterior. Revisar el panel Frames y hacer clic en el frame superior para ver las variables del metodo actual.

El SQL de Hibernate no aparece en la consola. Agregar las tres lineas de configuracion del apartado 3.2 (`show-sql`, `format-sql`, `logging.level`) y **reiniciar** el servidor en modo Debug. Los cambios en `application.yml` no se recargan en caliente.

El depurador es demasiado lento (F7 entra en clases internas de Spring). Usar **F8 (Step Over)** para avanzar sin entrar en los metodos internos. Solo usar **F7 (Step Into)** cuando se quiera entrar en una clase propia del proyecto. Para salir de una clase interna de Spring de regreso al codigo propio, usar **Shift+F8 (Step Out)**.

Conexion con la Tarea de practica fuera de clases

Siguiente paso: diagrama de secuencia UML

La tarea de practica fuera de clases (formativa) que sigue a esta practica consiste en convertir la tabla de trazado completada hoy en un **diagrama de secuencia UML** con los nombres reales de las clases del PFC. El diagrama debe mostrar los 9 pasos con los actores: Angular, JwtAuthFilter, DispatcherServlet, [NombreControlador], [NombreServicio], [NombreRepositorio] y PostgreSQL.

Herramientas recomendadas: **PlantUML** (texto plano, facil de versionar en Git), **draw.io** (visual, exporta PNG), **Mermaid** (integrado en GitHub). El diagrama se sube al repositorio en `/docs/diagramas/flujo-mvc-springboot.png`.